Class: 32 30/05/2025

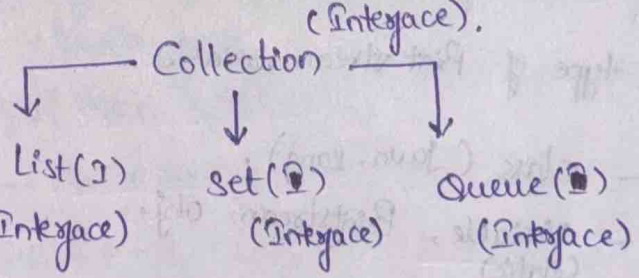
Collection Framework

* Collection is an interface.
* It has many advanced versions.
* Iterable is the parent of collection.
* This accepts heterogeneous (different) datatypes.
** It doesn't work with primitive datatypes (we use wrapper classes).
* It works with only classes, do not have fixed size.

Ex:
```
package com;

public class Array {
    public static void main [String[] args] {
        // size is Mandatory
        // Size is fixed
        // Homogeneous
    } To overcome this collections are introduced.
    int [ ] arr = new int [10];
}
```

* List ~~extends~~ collection.
* set extends collection. [ctrl + shift + t] - Class explanation.
* Queue extends collection.



List:

* It acts as an Arrays.
* This accepts duplicates.
* This preserves insertion order (maintains index i.e, order).
* It extends collection interface.

set:

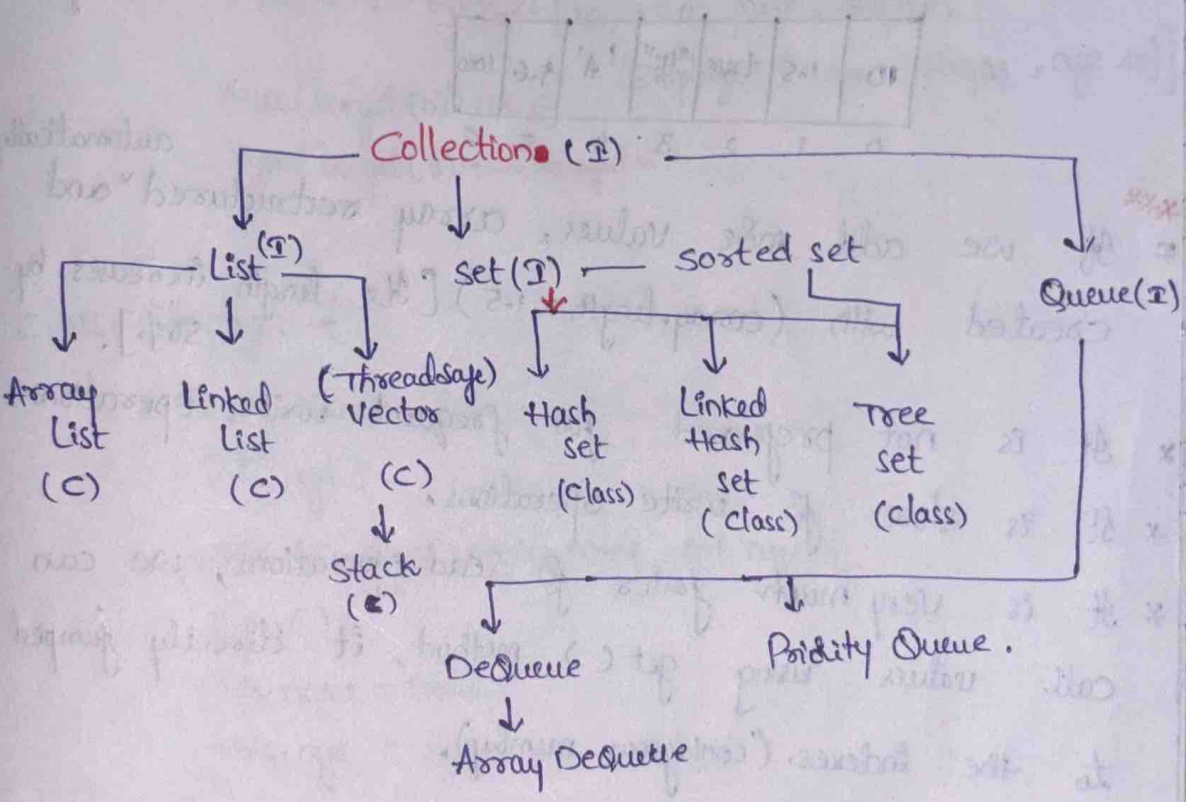
* This doesn't accept duplication. (internally overrides the same values).
* This doesn't follow insertion order. [there is no indexing]
  → It will store in random order.
* It extends collection interface.

Queue:

FIFO – First In First Out basis.

Stack:

First In Last Out. basis.



* Collections is a class in Java.

Vector:

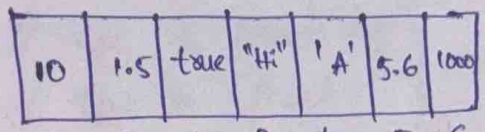
* Thread Safe
* Synchronized
* Slow.

* List : child of collection.

* Child classes : 1. Array list
  2. Linked list
  3. Vector.

1. Array List:

* It accepts Heterogeneous datatypes.
* It doesn't work with primitive datatypes, works with Wrapper class.
* It doesn't have fixed size.
* It works with only classes.
* It internally creats an Array (default size is 10), when we create an Array list.



***
* If we add more values, array restructured automatically and created with (array.length x 1.5) [Its length increases by 50%].
* It is not preferred for frequent write operations.
* It is slow for write operations.
* It is very much faster for read operations, we can call values using get() method, it directly jumped to the indexes. (Contageous memory).

* Syntax:

```
Arraylist objname = new Arraylist ( ) ;
```

Ex: package com;
java.
public class ArrayList Test {
public static void main(String [ ] args) {

ArrayList li = new ArrayList ( );
li.add (10); // to add data into list.
li.add (25.0); // It will take index automatically.
li.add (false);
li.add ("Gayatri");

output:
[10 , 25.0 , false , Gayatri]

* Syso (li);
4
* Syso(li.size());
* li. add (new Employee ("Ravi", 29));
Syso(li); output: [10, 25.0, false, Gayatri,
Employee [name = Fayaz , age 40]]
Syso(li.get(0)) :→ 10
Syso(li. get(3)); → Gayatri.

class Employee {
String name;
int age;
public Employee (String name , int age) {
Super( );
this.name = name;
this. age = age;
}
@Override
public void to String ( ) {

} Getters & Setters.

}

li.add(1, 20); // [If values are added, the
Index value all values are rearrangeled.]
syso(li).

Output: [10, 20, 25.0, false, Gayatri ---]

// when we try to change values, all values be reshuffled. (It takes lot of time internally).

* li.set(1, 30): // replace the value (it will not reshuffle the values.
syso(li)

Output: [10, 30, 25.0, false, Gayatri, Employee [Ravi, 29]]

* li.remove(1); // remover the value & reshuffle the values.

Output: [10, 25.0, false, Gayatri, ---]

* syso(li.isEmpty()); output: false.
* syso(li.size());

ArrayList a2 = new ArrayList();
syso(a2.isEmpty()); output: true.

* syso(li.contains("Java")); output: false.

* syso(li.indexOf("Gayatri")); output: 3
li.add(4, "Gayatri");

* syso(li.indexOf("Gayatri"); 3 (If same values are there in indexes, it return the 1st possible index)

* syso(li.lastIndexOf("Gayatri"); 4

data in li added to a2 list] ← * a2.addAll(li);

* li.clear() // it will clear all the data in the list.

syso(li); output: [ ] → Empty list.

Syso(li.isEmpty) output: true

Syso(li.size()) output: 0

Generics:

* This is used to define the datatype i.e used for collection
* <datatype>.
* this will remove heterogeneous functionality.
* It changes to Homogeneous and it will accept only same datatype (single datatype).
* ~~Array~~ Syntax: <E> → accepts all types of datatypes.

```
Arraylist<Integer> numbers = new ArrayList<Integer>();
numbers.add(10);
numbers.add("Gayatri"); [we can't add strings].
(error).                (∵ It accepts same type)

Arraylist<Employee> employee = new Arraylist<Employee>();
employee.add(new Employee("Ravi", 25);
employe.add(new Employee("FLM", 20);
Syso(employees.get(0));
```

output: Employee [name = Ravi, age = 29];

```
Syso(employee.get(0).name);
```

output: Ravi.

Class: 33 02/06/2025

Ex:

```
package com;
public class Iteration {
    public static void main(String[] args){
        Student st1 = new Student(1, "Ravi");
        Student st2 = new Student(2; "Sri");
        Student st3 = new Student(3; "Sai");
        Student st4 = new Student(4; "Fayaz");
    ArrayList students = new ArrayList();
Arraylist<Student> students = new Arraylist<Student>();
students.add(st1);
students.add(st2);
students.add(st3);
students.add(st4);
    syso(students);
}
class Student {
    int id;
    String name;
public student() {
}
public student(int id; String name){
    this.id = id;
    this.name = name;
}
// Getters & Setters.
```

* for (Student student : students){
    syso(student);
  }

Output:
Student [id=1; name=Ravi]
Student [id=2; name=Sri]
Student [id=3; name=Sai]
Student [id=4; name=Fayaz]

* for(int i=0; i<students.size(); i++){
    syso(students.get(i));
  }

Iterator:

* It works as a for loop, it works with all collections
// ~~students.iterator()~~ only in forward operation.

* Iterator<Student> iterator = students.iterator();
// has.Next(); → It takes if there is some value after this value in the list.

[st1, st2, st3, st4] → gives boolean values.
→ 0 1 2 3

while (iterator.hasNext()) {
  Student ~~next~~ student = iterator.next();
  syso(student);
}

Output:
Student [id=1; name=Ravi]
Student [id=2; name=Sri]
Student [id=3; name=Sai]
Student [id=4; name=Fayaz].

*** * Iterator moves forward with the help of next() method only.

* List Iterator:

* This is used for only lists.
* It has both sides iteration (can go forward & come backward).
* // has previous() & has Next().
(backward) (forward)

ListIterator<Student> listIterator = students.listIterator();
syso(listIterator.next())
syso(listIterator.next())
syso(listIterator.previous())
syso(listIterator.previous())
syso(listIterator.hasPrevious());

Output:
Student [id=1; name=Ravi]
Student [id=2; name=Sri]
Student [id=2; name=Sri]
Student [id=1; name=Ravi].
True.

Ex: package com;
public class SetOperation {

* Set:
* It is an interface.
* It also works as an ArrayList.
* It does not allow duplicates.
* It does not maintain order (there is no indexing internally).
* [Due to this we can't ~~each~~ use traditional for loop then we can use foreach loop]
* It uses internally Maps, to store values.

Ex:
Hashset:

* It doesn't allow duplicates.
  It doesn't maintain order.

Treeset:

* It doesn't allow duplicates
* It sets the data in ascending order.
* Sorted set is the parent of Tree set.

Linked Hashset:

* It doesn't allow duplicates
* It preserves (stores) order.

ex:

```
package com;
import java.util.Hashset;
public class setOperation{
    Psvm {
HashSet<Integer> nums = new Hashset<Integer>();
nums.add(10);
nums.add(20);   [∵ doesn't allow duplicates]
nums.add(15);
nums.add(20);
* for(Integer num : nums){
        Syso(num);
  }
```

Output:
[20, 10, 15]

```
* Syso(nums);
```

[20, 10, 15]

Output:
20
10
15

*

```
Iterator<Integer> iterator = nums.iterator();
  while(iterator.hasNext()){
      Syso(iterator.next());
  }
```

Output:
20
10
15